

CS6111 (2025): Assignment 3

Instructor: John Augustine

Due: Oct. 26, 2025 (11 PM)

- Use $\LaTeX$ to typeset your answer. You must submit the PDF file.
- You can use theorems/lemmas from the textbook (Katz and Lindell, Edition 3 only) without proving them. However, you should mention the theorem/lemma number from the textbook and paraphrase the statement of the theorem/lemma to make your solution self-contained.
- The deadline is Oct. 26, 2025 (11 PM).
- Please follow the academic honesty policy for assignments. If you are stuck and need a hint, please post a question on Moodle.
- The assignment is 24 marks. It will be later scaled down to 6 marks.

1. (0 marks) Reading assignment: Katz and Lindell Ed. 3: §6.5, §6.6, §9.1, §9.3 (starred sections can be skipped).

Solution: Yes, I have fully read sections 6.5, 6.6, 9.1, 9.3

2. (5 marks) Explain the importance of padding and encoding message length in hash function constructions. Show with a concrete counterexample how omitting message length encoding can result in collisions even if the compression function is collision-resistant.

Solution:

Hash functions typically operate on messages of arbitrary length by dividing them into fixed-size blocks and processing them iteratively using a compression function.

Let the message m be divided into blocks $x_1, x_2, \dots, x_n$ after padding, and let the iterative construction be defined as

$$z_0 = IV, \quad z_t = h(z_{t-1} \| x_t) \text{ for } t = 1, 2, \dots, n,$$

with the final hash given by $H(m) = z_n$.

Importance of Padding:

Padding ensures that:

1. The message length becomes a multiple of the block size, so the iteration can process complete blocks.
2. The padding is **unambiguous**, i.e. no two different messages should produce the same padded sequence of blocks.

A common padding rule is to append a ‘1’ bit followed by the minimum number of ‘0’s to fill the final block.

If we use a technique such as appending all 0’s, then if the scheme has say block length of 3, $m_1 = 10$, $m_2 = 1$, then both would become 100 after padding leading to collision.

However, padding with a standard scheme alone is insufficient to prevent collisions between messages of different lengths and **encoding the message length** (usually in the final block) is critical.

Importance of Encoding the Message Length:

Encoding the message length ensures that two messages of different length (for example, one being an extension of another) cannot produce the same internal chaining state at some block followed by similar blocks for both messages giving same final hash. Without this step, certain iterative constructions become *length-extension vulnerable*.

To see why, consider two messages m and m' , where

$$m' = m'' \| m.$$

Suppose that $H(m'') = IV$, i.e., the internal state after processing m'' equals the initial value. (We take m'' that is padded to the block length multiple). Then,

$$H(m') = H(m'' \| m) = H(m),$$

since processing m'' effectively resets the internal state to IV before processing m . Thus, m and m' produce the same hash output even though they are distinct messages.

This collision arises because the iterative construction has no way to distinguish between a message that starts from the initial state IV and one that returns to that state midway through processing. Encoding the total message length in the final block prevents this ambiguity, since m and m' would now have different encoded lengths and thus different final blocks.

Similarly, padding through standard scheme like adding 1 followed by 0's could give same padded message without length encoding. For example, if block length is 3, messages $m_1 = 1$, and $m_2 = 110$ both after padding become 110. Instead encoding message length makes the padded messages as 110001, 110111 which are distinct.

Hence, both padding and message length encoding are essential for the collision resistance of any iterative hash construction.

3. (3 marks) Fix $H : \{0,1\}^n \rightarrow \{0,1\}^{2n}$, and define the keyed function $F : \{0,1\}^n \times \{0,1\}^n \rightarrow \{0,1\}^{2n}$ by

$$F_k(x) = H(k \oplus x).$$

Show that an attacker given oracle access to $F_k(\cdot)$ can recover the n -bit key k with constant probability in time approximately $2^{n/2}$.

Solution:

$F_k(x) = H(k \oplus x)$ where $H : \{0,1\}^n \rightarrow \{0,1\}^{2n}$.

Algorithm:

1. Compute $Q = \lceil \sqrt{2^n} \rceil \approx 2^{n/2}$ random hash values:

$$\mathcal{C} = \{(H(y_j), y_j) : y_j \leftarrow \{0,1\}^n, j = 1, \dots, Q\}$$

2. Query oracle for random inputs until a collision is found: Look for (x_i, y_j) such that:

$$F_k(x_i) = H(y_j)$$

This implies $H(k \oplus x_i) = H(y_j)$.

3. With high probability, $k \oplus x_i = y_j$, so:

$$k = x_i \oplus y_j$$

4. Verify candidate k with additional oracle queries to find more collisions

Probability analysis:

The problem is similar to balls and bins problem. We have 2^n possible outputs to H (though collisions in H would reduce this) of which we have $2^{\frac{n}{2}}$ stored in the hash table. When we query the function F , we get outputs among the 2^n possible hash outputs which are the N bins. Now let number of queries to F be M (number of balls). We need to find number of balls so that atleast one ball falls into any of the $2^{\frac{n}{2}}$ previous hashed values with probability c .

$$(1 - (1 - \frac{2^{n/2}}{2^n})^M) = c$$

$$(1 - \frac{1}{2^{n/2}})^M = 1 - c$$

$$(\exp^{\frac{-M}{2^{n/2}}}) = 1 - c$$

$$\frac{-M}{2^{n/2}} = \ln(1 - c)$$

The probability of at least one collision between $\mathcal{C}$ and queries is c when:

$$M \approx 2^{n/2} \ln \frac{1}{1 - c}$$

When m is of order $2^{n/2}$ we have a constant probability of finding a collision of F and H .

Time complexity:

- $M \approx 2^{n/2}$ oracle queries
- $Q \approx 2^{n/2}$ offline computations
- $O(2^{n/2})$ storage and comparison time

4. (3 marks) Given a hash function (Gen, h) that is collision resistant, prove or disprove whether the following hash functions (Gen, H) are also collision resistant:

- (a) $H(x) = h(x \mid 1)$ (bitwise OR)

Solution:

Let us take 2 messages that differ only at their last bit. Let $x_1 = x' \parallel 1$ and $x_2 = x' \parallel 0$, where x' is one bit less than x_1 and x_2 . Now both $x_1 \mid 1$ and $x_2 \mid 1$ are equal to x_1 . (Last bit on ORing with 1 gives 1)

Thus though $h(x_1) \neq h(x_2)$ for $x_1 \neq x_2$ with probability $\geq 1 - \text{negl}(n)$, $H(x_1) = H(x_2)$.

Thus we can find a collision in H .

The probability of getting a collision for $x \neq x'$ in H , is choosing

$$\frac{\frac{2^n}{2}}{2^n C_2} = \frac{1}{2^n - 1} \geq \frac{1}{2^n}$$

Thus, for length n varying from 1 to ∞ , we have Probability of collision to be

$$\geq \sum_{n=1}^{n=\infty} \frac{1}{2^n} \geq \int_{n=1}^{n=\infty} \frac{1}{2^n} = \frac{1}{2 \ln 2}$$

Thus we can get a colliding pair with a constant probability and

$$\Pr[\text{Hash-coll}_{\mathcal{A}, \Pi}(n) = 1] \leq \text{negl}(n)$$

is disproved. Thus it is not collision resistant.

- (b) $H(x) = h(x \parallel \text{LSB}(x))$ (concatenation, where LSB denotes the least significant bit)

Solution:

Let us consider a function that takes an input, x and appends $\text{LSB}(x)$ to it.

$$f(x) = x \parallel \text{LSB}(x)$$

This function is one to one. If $x_1 \neq x_2$ then $x_1 \parallel \text{LSB}(x_1) \neq x_2 \parallel \text{LSB}(x_2)$ as they differ in their first n bits where $n+1$ is the length of function value. If $x_1 = x_2$, then $\text{LSB}(x_1) = \text{LSB}(x_2)$, thus $f(x_1) = f(x_2)$, and $f(\cdot)$ is one-one.

Thus if h is collision resistant and no 2 different inputs can give the same value in range for f . Thus Probability of finding a collision in H is same as the probability of finding a collision in h (As it maps in an injective manner).

$$\Pr[\text{Hash-coll}_{\mathcal{A}, \Pi}(n) = 1] \leq \text{negl}(n)$$

Thus if h is collision resistant, H is also collision resistant.

5. (5 marks) Consider the modification of the Merkle–Damgård transform where the initial chain value, i.e. z_0 , is the length L of the input string x instead of a fixed IV. Given that (Gen, h) is collision resistant, state and prove whether (Gen, H) is collision resistant or not.

Solution: Let us consider 2 cases where L and L' are the lengths of 2 different messages m and m' .

Let us consider that the length is given as IV and that both padding and length encoding is present at the end of message. We cascade output at each step, i as

$z_i = h(z_{i-1}, x_i)$ if x_i is the i th block of message and $z_0 = t$.

Case 1: $L = L'$

If the lengths are equal we give the same IV=L for both message blocks. However the same padding is given and length L common to both messages is appended along with padding at the end. But the messages differ at some block k, where k is the largest block with different blocks for both messages (As $m \neq m'$, there exists $x_k \neq x'_k$)

$$h(z_{k-1} || x_k) \neq h(z'_{k-1} || x'_k)$$

with high probability as (Gen, h) is collision resistant. Thus it is collision resistant in messages of same length

Case 2: $L \neq L'$

As the last block in Merkle-Damgard construction would have the length appended, if $L \neq L'$ and (Gen, h) is collision resistant, then we cannot have a collision with high probability (To have a collision, we would need $h(z_n || L)$ to be equal to $h(z'_n || L')$ for $L \neq L'$ and $n \neq n'$)

Therefore, the modified construction is collision resistant.

6. (4 marks) Describe a man-in-the-middle attack on the Diffie-Hellman protocol where the adversary shares a key k_A with Alice and a (different) key k_B with Bob, and Alice and Bob cannot detect that anything is wrong. Show why your attack works.

Solution:

Setup:

- Public parameters: cyclic group G of order q , generator g
- Alice's private key: $a \leftarrow Z_q$
- Bob's private key: $b \leftarrow Z_q$
- Eve's private keys: $e_A, e_B \leftarrow Z_q$

Attack execution:

1. **Alice sends Bob:** $A = g^a$
Eve intercepts and sends to Bob: $A' = g^{e_A}$
2. **Bob sends Alice:** $B = g^b$
Eve intercepts and sends to Alice: $B' = g^{e_B}$

3. Key computation:

Alice computes: $k_A = (B')^a = g^{e_B a}$

Bob computes: $k_B = (A')^b = g^{e_A b}$

Eve's computation of Alice's key: $k_A = A^{e_B} = g^{a e_B}$

Eve's computation of Bob's key: $k_B = B^{e_A} = g^{b e_A}$

- Alice shares key k_A with Eve
- Bob shares key k_B with Eve
- Alice and Bob believe they share a key with each other

All messages A, A', B, B' are valid group elements and the protocol flow appears normal. Without authentication, Alice and Bob cannot verify each other's identity

The attack succeeds perfectly.

7. (4 marks) Define a MAC for arbitrary-length messages by

$$\text{MAC}_k(m) = H(k\|m)$$

where H is a collision-resistant hash function.

Show that this is not a secure MAC when H is constructed via the Merkle–Damgård transform. Assume $k \in \{0, 1\}^n$.

Solution:

Let the Merkle–Damgård construction use:

- Compression function $h : \{0, 1\}^n \times \{0, 1\}^\ell \rightarrow \{0, 1\}^n$
- Padding function $\text{pad}(\cdot)$
- Initial value IV

The legitimate MAC computation is given by

$$\begin{aligned}\text{MAC}_k(m) &= H(k\|m) \\ &= z_t\end{aligned}$$

where we have cascaded output at each step, i.e. as $z_i = h(z_{i-1}, m_i)$ if m_i is the i th block.

The attacker obtains $z_t = \text{MAC}_k(m) = H(k||m)$ after appending p followed by L where p is the pad for message, m under the padding scheme, pad(.) and L is the length of the message

With access to $H(\cdot)$ but not key, k we could compute

$$\text{MAC}(m||p||L||w) = H(k||m||p||L||w)$$

by invoking MAC for m first to get z_t and then finding $h(z_t, w)$ if w is formed after padding and length appending according to Merkle-Damgård.

Else find length L' and pad p' of $k||m||p||L||w$ and then cascade outputs from $z_{t+1} = h(z_t, w)$ into subsequent blocks of message having p', L' . Thus the attacker successfully forges $\text{MAC}_k(m||p||L||w||p'||L')$ without knowing k and IV

[The adversary has access to neither, k nor $H^{-1}(\cdot)$ in an ideal setup. We do not know IV either to query $\text{MAC}(\cdot)$ for $2^{\frac{n}{2}}$ queries to find the preimage of IV. Else we could find preimage of IV and assign, it to message m and query w to $H(\cdot)$. This gives the value of MAC for $m||w$.]

The MAC is insecure when H uses Merkle-Damgård due to length-extension vulnerability while having access to the compression function, h.

Acknowledgment

I would like to thank Swathi Shree N for their useful discussion on solving the problems.